



Universidad de Jaén

Bloque 1

Software e Ingeniería del Software.

Una Introducción



EPS
Escuela Politécnica
Superior de Jaén

Tema 1. Software e Ingeniería del Software

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

1 Objetivos

2 La naturaleza del software

- El papel evolutivo del software
- Motivación

3 Software

- Definición
- Dominios de aplicación del software
- Software heredado
- Mitos del software

4 Ingeniería del software

- Definición
- Tecnología de capas

Objetivos

- Valorar la **importancia** del **software** en la actualidad
- Comprender el **concepto** de **software**
- Descubrir las **características** del **software**
- Conocer las categorías de **aplicación**
- Conocer las **curvas de fallo** y el concepto de **deterioro** del **software**
- Conocer las especiales características del **software heredado**
- Discutir sobre los **mitos** del **software** y sus efectos
- Conocer en qué consiste la **Ingeniería del Software** y su singularidad como **Tecnología de Capas**

El papel evolutivo del software

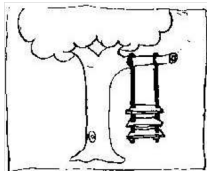
- Todas las **economías** de los países desarrollados **dependen** del software
- La mayoría de los **sistemas** están **controlados** por el software
- El **presupuesto** destinado al software representa una **importante** fracción del PIB de los **países desarrollados**
- El software permite la **automatización** de los **procesos** desarrollados en negocios, industrias, organismos y gobiernos
- Es un medio para **transferir** tecnología, experiencia o para compartirla
- El software puede ser utilizado como un **elemento diferenciador** entre distintas compañías

El papel evolutivo del software

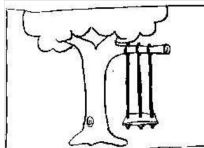
A pesar de los grandes avances se siguen haciendo las mismas **preguntas**:

- ¿Por qué **tarda** tanto la obtención del software?
- ¿Por qué son **altos** los **costes** de desarrollo?
- ¿Por qué es **imposible encontrar** todos los **errores** antes de la entrega al cliente?
- ¿Por qué se invierte tanto **tiempo** en el **mantenimiento** de programas existentes?
- ¿Por qué es **difícil medir** el **progreso** al desarrollar y mantener?

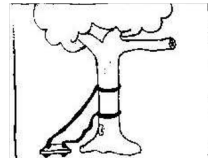
Motivación



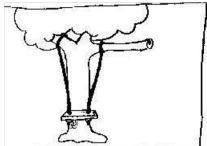
Lo que ofrece el vendedor



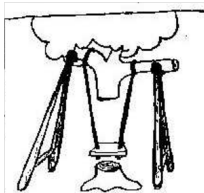
Lo que recomienda el asesor



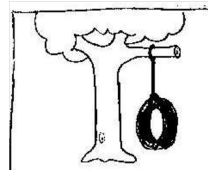
Lo que entiende el analista



Lo que hace el programador



Lo que se usa al final



Lo que quería el usuario

Motivación



Informe CHAOS 2013 (Standish Group)

- En **2012**, el **18 %** de los proyectos fueron **cancelados** por algún motivo y el **43 %** sufrieron **retrasos en tiempo, incrementos en costo o pérdida de requisitos**
- Únicamente el **39 %** restante terminó en el **tiempo, costo y requisitos convenidos**
- Los proyectos finalizados en **2012** sufrieron en media un **retraso en su finalización del 74 %**
- Los proyectos finalizados en **2012** sufrieron en media un **incremento de coste del 59 %**
- Los proyectos finalizados en **2012** recogieron en media un **69 % de los requisitos solicitados**

Motivación

- Los **problemas** que se presentan en la construcción de **grandes sistemas software** no son simples **versiones** a gran escala de los problemas de escribir **pequeños programas**



La complejidad de los grandes sistemas requiere **trabajar en grupo**, utilizar **técnicas** más rigurosas de **análisis y diseño**, **documentar** las decisiones, **pruebas** exhaustivas y cuidadosa **administración**

¿Por qué fracasan los proyectos software?



Baja calidad de los sistemas construidos

- Solución al problema equivocado
 - El sistema entra en conflicto con la estrategia de negocio
- Falta consideración influencias más amplias
 - Se ignora la cultura de la organización
- No se analiza bien el problema
 - El equipo no está preparado o no dispone de recursos suficientes
- Se realiza el proyecto por una razón errónea
 - Empuje de tecnología (e.g. Fracaso de las .com o burbuja tecnológica) u obligación política

¿Por qué fracasan los proyectos software?



Baja productividad en desarrollo, tiempo y/o recursos consumidos

- Cambio en los requisitos
 - Usuario cambia de idea
- Eventos externos cambian el entorno
 - E.g. Cambio de legislación, tarifas o disponibilidad de servicios, etc.
- La implementación no es posible
 - Exceso de ambición, o de difícil integración con sistemas previos ya existentes
- Pobre control del proyecto
 - Gestor de proyecto inexperto: proyecto no acaba a tiempo, costes excesivos, etc

¿Cómo solucionar los problemas asociados a la baja calidad del sistema construido?

- Definiendo **modelos (entregables)** que permitan comprender mejor el sistema y ayuden a incrementar las posibilidades de éxito del sistema software resultante
- Definiendo **modelos de desarrollo de software** a partir de los cuales se instancian **procesos** que subdividen los pasos a realizar para el desarrollo del software y de este modo ayudan a manejar la complejidad del proyecto



¿Qué es un Proceso?

Instancia particular de un **modelo de desarrollo** adaptado a una organización y un **proyecto** de desarrollo **concreto**

¿Qué es el **software**?

El **software** se considera formado por:

- 1 **Instrucciones** (programas de ordenador) que cuando se ejecutan proporcionan las características, función y desempeño buscados
- 2 **Estructuras de datos** que permiten que los programas manipulen de forma adecuada la información
- 3 **Información descriptiva** tanto en papel como en formas virtuales que describen la operación y uso de los programas

Doble misión del software

- ❶ Por un lado es un **producto**
 - Como producto el software pone a disposición las posibilidades de un ordenador o de una red de ordenadores
 - Estas posibilidades pueden ser las de producir, manejar, adquirir, modificar, desarrollar o transmitir información
- ❷ Por otro lado es una **herramienta** para el desarrollo de un producto
 - Soporta o facilita la **funcionalidad** de un **sistema**
 - Controla el ordenador (**Sistema Operativo**)
 - Ofrece la comunicación entre ordenadores (**redes**)
 - Ayuda en la elaboración de otro software (**herramientas**)

Hardware vs Software

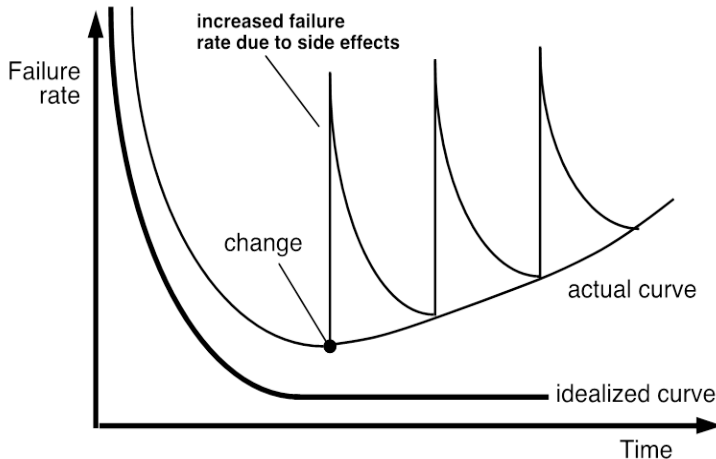
- Para construir **hardware** el proceso creativo humano se traduce en una **forma física**
- En cambio, el **software** es un elemento del sistema que tiene un **carácter lógico** y no físico



Características

- El software **se desarrolla**, no se fabrica en un sentido clásico
- El software **no se desgasta**
- Aunque la industria se mueve hacia la construcción basada en componentes, la mayor parte del software se construye para un **uso individualizado**

Curva de fallos del software



Dominios de aplicación del software

- **Software de sistemas:** conjunto de programas escritos para dar servicio a otros programas (compiladores, editores, administración de archivos, componentes de sistemas operativos, software de redes, de telecomunicaciones, etc). Presenta gran interacción con el hardware, posibilidad de usuarios múltiples, concurrencia, gestión de recursos e interfaces externas múltiples
- **Software de aplicación:** programas independientes que resuelven una necesidad de negocios específica
- **Software científico y de ingeniería:** aplicaciones en astronomía, vulcanología, análisis de tensiones en automóviles, dinámica orbital en transbordador espacial, biología molecular o fabricación automatizada, etc

Dominios de aplicación del software

- **Software empotrado**: reside dentro de un producto o sistema y se usa para el control de características específicas de ese producto: control del teclado de un horno microondas o programación de las funciones de un automóvil (control del combustible, sistema de frenado, ordenador de a bordo, etc)
- **Software de línea de productos**: puede estar centrado en un mercado limitado (p.e. control del inventario de productos) o en un mercado masivo de consumidores (procesadores de textos, hojas de cálculo, gestores de bases de datos, multimedia, entretenimiento, etc)

Dominios de aplicación del software

- **Aplicaciones Web y Móviles**: concentra un amplio abanico de aplicaciones, tanto las basadas en el uso del navegador como aquellas desarrolladas para dispositivos móviles
- **Software de inteligencia artificial**: Se caracteriza por el uso de algoritmos no numéricos para la resolución de problemas complejos: robótica, sistemas expertos, reconocimiento de patrones (imagen y voz), redes neuronales artificiales, demostración de teoremas y juegos

Características

- El software heredado está caracterizado por su **longevidad** y por ser críticos en los negocios
- En muchas ocasiones este software tiene diseños imposibles de entender, **código complicado, escasa documentación, casos de prueba no archivados, etc**

Necesidad de evolución

El software heredado evoluciona por varios motivos:

- Debe **adaptarse** para satisfacer las necesidades de nuevos ambientes o **nuevas tecnologías**
- Debe mejorarse para implementar **nuevos requisitos**
- Debe extenderse para operar con **otros sistemas o bases de datos** modernos
- La arquitectura del software debe rediseñarse para hacerla viable dentro de un ambiente de **redes**

¿En qué consisten?

- Los mitos del software afectan a los gestores y administradores, a los usuarios y a los desarrolladores
- Suelen ser aceptados puesto que contienen **algún** elemento de **verdad**
- Aunque en realidad siempre nos **encaminan** hacia **decisiones erróneas**

Mitos de gestión y administración

- Ya **tenemos** un libro de **estándares y procedimientos** para el desarrollo de software. Esto proporcionará el conocimiento **suficiente** a mi gente
- Tenemos las **computadoras más modernas** con lo que nuestro software será de **mayor calidad**
- Si vamos con **retraso** en un proyecto siempre podemos añadir **más programadores** para terminar el trabajo a tiempo

Mitos del cliente

- Una declaración inicial de objetivos es suficiente para comenzar a escribir programas, los **detalles** se pueden refinar **después**
- Los **requisitos** del proyecto son **cambiantes** pero éstos pueden ser **ajustados** con **facilidad** dada la flexibilidad del software

Mitos del desarrollador

- Una vez escribimos el **programa** y **funcionando** nuestro **trabajo** ha **terminado**
- No puedo **probar** hasta tener el ejecutable
- Lo único que se **entrega** es el **programa funcionando**
- La **ingeniería del software** obligará a crear una voluminosa e innecesaria documentación que hará que el proceso de **desarrollo** sea **más lento**

Retos del software del siglo XXI

- Cada vez es **mayor** el **número** de **personas** que tienen interés en las características y funciones de una aplicación específica
 - Es necesario un **esfuerzo** concertado para **entender** el **problema** antes de desarrollar una aplicación de software
- Los **requisitos** de la tecnología de la información que demandan individuos, negocios y gobiernos se hacen cada vez **más complejos**
 - El **diseño** se ha vuelto una **actividad crucial**

Retos del software del siglo XXI

- Los individuos, negocios y gobiernos dependen cada vez más del **software** para tomar **decisiones estratégicas y tácticas**
 - El software debe tener **alta calidad**
- Conforme aumenta el valor percibido de una aplicación se **incrementa** la **probabilidad** de que su base de usuarios y **longevidad** crezcan
 - El software debe **facilitar el mantenimiento**



Conclusión

Debe hacerse ingeniería con el software en todas sus formas y a través de todos sus dominios de aplicación. Esto nos lleva a la **Ingeniería del Software**

Definiciones



Fritz Bauer, 1969

La Ingeniería del Software es el **establecimiento y uso** de principios fundamentales de la ingeniería con objeto de desarrollar de forma **económica** software que sea **fiable** y que trabaje **eficientemente** en máquinas reales



IEEE, 1993

La Ingeniería del Software es:

- 1 La aplicación de un **enfoque sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento de software; es decir, la aplicación de la ingeniería al software
- 2 El **estudio de enfoques** según el punto anterior

Tecnología de capas



Ingeniería del Software

Bibliografía



R. Pressman

Software Engineering. A Practitioner's Approach. 8th Ed.

McGraw-Hill, 2014